

UD3 - Prácticas: criptografía

Hashes, cifrado, firmas, certificados y PKI en un entorno Linux de pruebas.

Datos de las prácticas	Información
Módulo	Seguridad y Alta Disponibilidad
Curso	2.º ASIR
Modalidad	Semipresencial
Duración estimada	10 horas
Entorno	Máquina virtual Linux con OpenSSL y GnuPG

1. Objetivos

Al finalizar el itinerario, el alumnado será capaz de:

- Comprobar la integridad de ficheros mediante SHA-256.
- Generar y proteger pares de claves RSA en un entorno de pruebas.
- Crear solicitudes de certificado, certificados autofirmados y una CA de entorno de pruebas.
- Firmar y verificar un certificado de servidor.
- Utilizar GnuPG para cifrar y firmar datos de prueba.
- Analizar de forma autorizada la fortaleza de contraseñas de cuentas de prueba.
- Comprobar los elementos principales de una conexión TLS.
- Documentar las evidencias sin divulgar claves privadas.

2. Alcance y preparación

Trabaja únicamente en una máquina virtual propia y con datos de prueba. Las claves y certificados generados en esta guía son exclusivos del entorno de pruebas: no deben utilizarse en producción ni para proteger información real. No publiques claves privadas, archivos **.key** ni contraseñas.

1. Actualiza el sistema e instala las herramientas:

```
# AlmaLinux, Rocky Linux o Fedora
sudo dnf install openssl gnupg2 -y

# Debian o Ubuntu
sudo apt update
sudo apt install openssl gnupg -y
```

2. Crea el directorio de trabajo y restringe su acceso:

```
mkdir -p ~/ud3-criptografia/{ca,servidor,datos}
chmod 700 ~/ud3-criptografia
```

```
cd ~/ud3-criptografia
```

En los ejemplos de esta guía se utilizará la identidad ficticia **Lorena L Resusta** <lorena.resusta@ejemplo.com>. Sustituye los datos de ejemplo por los tuyos cuando una actividad lo solicite.

3. Práctica 1 - Integridad con funciones hash

3.1. Calcular y comprobar un hash

1. Crea un fichero de prueba y calcula su resumen:

```
echo "Mensaje de prueba" > datos/mensaje.txt  
sha256sum datos/mensaje.txt | tee datos/mensaje.txt.sha256
```

2. Comprueba el resumen almacenado:

```
sha256sum -c datos/mensaje.txt.sha256
```

3. Modifica el fichero y vuelve a comprobarlo:

```
echo "Mensaje modificado" > datos/mensaje.txt  
sha256sum -c datos/mensaje.txt.sha256
```

Explica por qué la segunda comprobación falla y por qué un hash permite detectar cambios, pero no recupera el contenido original.

3.2. Verificar una imagen ISO

Descarga una imagen ISO y su archivo oficial de sumas de comprobación, por ejemplo desde AlmaLinux o Debian. Calcula el SHA-256 del archivo y compáralo con el valor publicado por el proyecto.

```
# Linux  
sha256sum nombre-de-la-imagen.iso
```

```
# Windows PowerShell  
Get-FileHash .\nombre-de-la-imagen.iso -Algorithm SHA256
```

Incluye el nombre de la ISO, el algoritmo utilizado, el valor publicado y el valor calculado. No descargues imágenes desde fuentes no oficiales.

4. Práctica 2 - Claves RSA, cifrado y firma con GnuPG

4.1. Generar y proteger una clave RSA

1. Genera una clave privada RSA de entorno de pruebas y limita sus permisos:

```
cd ~/ud3-criptografia/servidor
openssl genrsa -out servidor.key 2048
chmod 600 servidor.key
```

2. Examina la clave sin copiar su contenido en la entrega:

```
openssl rsa -in servidor.key -check -noout
```

3. Obtén la clave pública asociada:

```
openssl rsa -in servidor.key -pubout -out servidor.pub
openssl pkey -pubin -in servidor.pub -text -noout
```

Anota qué archivo puede distribuirse y cuál debe permanecer protegido.

4.2. Cifrado simétrico con GnuPG

1. Crea un archivo confidencial de prueba y cífralo con una frase de paso que recuerdes:

```
cd ~/ud3-criptografia
echo "Documento confidencial de Lorena L Resusta" > datos/confidencial.txt
gpg --symmetric datos/confidencial.txt
file datos/confidencial.txt.gpg
```

2. Elimina el original de prueba y recupera una copia:

```
rm datos/confidencial.txt
gpg --output datos/confidencial-recuperado.txt --decrypt
datos/confidencial.txt.gpg
cat datos/confidencial-recuperado.txt
```

Explica qué ocurriría si se olvidara la frase de paso y por qué no debe enviarse junto al archivo cifrado.

4.3. Cifrado asimétrico y firma con GnuPG

1. Genera un par de claves de entorno de pruebas. Usa los datos de ejemplo **Lorena L Resusta** <lorena.resusta@ejemplo.com>, una caducidad de un año y una contraseña segura:

```
gpg --full-generate-key
gpg --list-keys
gpg --list-secret-keys
```

2. Exporta únicamente tu clave pública e importa la clave pública del profesor, suministrada como **fperez.asc**:

```
gpg --output datos/lorena-l-resusta-publica.asc --armor --export "Lorena L Resusta"
gpg --import fperez.asc
gpg --list-keys
gpg --fingerprint
```

Verifica la huella de la clave importada por un canal fiable antes de utilizarla. Sustituye **ID_PROFESOR** por el identificador o correo que muestre **gpg --list-keys**.

3. Crea un mensaje con tu nombre y la fecha, cífralo para el profesor y fírmalo con tu clave:

```
printf 'Mensaje creado por Lorena L Resusta el %s\n' "$(date +%F)" >
datos/mensaje-lorena.txt
gpg --armor --output datos/mensaje-lorena.asc --encrypt --sign \
  --local-user "Lorena L Resusta" --recipient ID_PROFESOR datos/mensaje-
lorena.txt
```

Como entrega de esta actividad, utiliza **mensaje-lorena.asc** y **lorena-l-resusta-publica.asc**. No incluyas la clave privada ni su contraseña. Describe la diferencia entre cifrar, firmar y cifrar junto con firmar.

5. Práctica 3 - Auditoría controlada de contraseñas

Esta práctica solo está autorizada sobre cuentas creadas expresamente en la máquina virtual propia. No copies, analices ni entregues hashes de cuentas reales o de otros sistemas.

5.1. Cuentas y almacenamiento de credenciales

1. Crea dos cuentas de prueba con contraseñas débiles conocidas exclusivamente para el entorno de pruebas:

```
sudo useradd -m lorena_lab
sudo passwd lorena_lab
sudo useradd -m pedro_lab
sudo passwd pedro_lab
```

2. Comprueba dónde se almacenan la información pública de usuarios y los hashes de contraseña, así como sus permisos:

```
getent passwd lorena_lab pedro_lab
sudo ls -l /etc/passwd /etc/shadow
sudo grep -E '^(lorena_lab|pedro_lab):' /etc/shadow
```

Explica la función de `/etc/passwd` y `/etc/shadow`, el significado general de los campos separados por `:` y por qué los hashes de dos usuarios con la misma contraseña pueden ser distintos.

5.2. Auditoría con John the Ripper

Instala John the Ripper desde los repositorios disponibles en tu distribución o utiliza la instalación proporcionada por el docente:

```
# Debian o Ubuntu
sudo apt install john -y

# AlmaLinux, Rocky Linux o Fedora con EPEL disponible
sudo dnf install epel-release -y
sudo dnf install john -y
```

Genera un archivo temporal limitado a las dos cuentas de prueba y analiza únicamente ese archivo:

```
sudo unshadow /etc/passwd /etc/shadow > /tmp/ud3-cuentas-completas.txt
sudo grep -E '^(lorena_lab|pedro_lab):' /tmp/ud3-cuentas-completas.txt > ~/ud3-criptografia/datos/ hashes-prueba.txt
john ~/ud3-criptografia/datos/ hashes-prueba.txt
john --show ~/ud3-criptografia/datos/ hashes-prueba.txt
rm -f /tmp/ud3-cuentas-completas.txt ~/ud3-criptografia/datos/ hashes-prueba.txt
```

Documenta el resultado sin incluir el archivo de hashes ni las contraseñas en la entrega. Relaciona la prueba con el uso de funciones de derivación como Argon2, bcrypt, scrypt o PBKDF2, con *salt* único y con políticas de contraseñas robustas.

6. Práctica 4 - CSR y certificado autofirmado

6.1. Crear una solicitud de certificado

1. Genera una CSR a partir de la clave del servidor. Cuando OpenSSL solicite el nombre común, utiliza `servidor.lan`:

```
cd ~/ud3-criptografia/servidor
openssl req -new -key servidor.key -out servidor.csr
```

2. Consulta su contenido:

```
openssl req -in servidor.csr -text -noout
```

Explica por qué una CSR contiene una clave pública, pero no debe contener la clave privada.

6.2. Crear un certificado autofirmado

1. Genera un certificado válido durante 365 días para pruebas:

```
openssl req -x509 -new -key servidor.key -out servidor-autofirmado.crt -days 365
```

2. Examina emisor, sujeto, validez y clave pública:

```
openssl x509 -in servidor-autofirmado.crt -text -noout
```

Indica por qué este certificado es apropiado para un entorno de pruebas, pero no genera confianza automática en los navegadores.

7. Práctica 5 - CA y certificado de servidor

7.1. Crear una CA de entorno de pruebas

1. Genera la clave y el certificado raíz de la CA:

```
cd ~/ud3-criptografia/ca
openssl genrsa -out ca.key 4096
chmod 600 ca.key
openssl req -x509 -new -key ca.key -sha256 -days 3650 -out ca.crt
```

2. Muestra solo la información pública del certificado:

```
openssl x509 -in ca.crt -subject -issuer -dates -noout
```

7.2. Firmar y verificar el certificado del servidor

1. Firma la CSR creada en la práctica anterior:

```
cd ~/ud3-criptografia
openssl x509 -req -in servidor/servidor.csr -CA ca/ca.crt -CAkey ca/ca.key \
```

```
-CAcreateserial -out servidor/servidor.crt -days 365 -sha256
```

2. Comprueba la cadena de confianza:

```
openssl verify -CAfile ca/ca.crt servidor/servidor.crt
openssl x509 -in servidor/servidor.crt -subject -issuer -dates -noout
```

El resultado de la verificación debe ser **servidor/servidor.crt: OK**. Conserva capturas de la creación de la CA, la CSR, el certificado firmado y la verificación.

7.3. Certificado con nombres alternativos y Apache

Para un certificado de servidor, los nombres válidos deben figurar en la extensión SAN. Crea el archivo **servidor/san.cnf**, sustituyendo el apellido si utilizas dominios propios:

```
[v3_req]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = www.servidorresusta.test
DNS.2 = cursoresusta.test
DNS.3 = www.cursoresusta.test
```

Firma de nuevo la CSR incluyendo la extensión y comprueba los SAN:

```
openssl x509 -req -in servidor/servidor.csr -CA ca/ca.crt -CAkey ca/ca.key \
  -CAcreateserial -out servidor/servidor-san.crt -days 365 -sha256 \
  -extfile servidor/san.cnf -extensions v3_req
openssl x509 -in servidor/servidor-san.crt -noout -ext subjectAltName
```

Para configurarlo en Apache, instala el servidor y el módulo TLS. Usa solo una de las alternativas según la distribución:

```
# AlmaLinux, Rocky Linux o Fedora
sudo dnf install httpd mod_ssl -y
sudo install -m 600 -o root -g root servidor/servidor.key
/etc/pki/tls/private/servidor-resusta.key
sudo install -m 644 servidor/servidor-san.crt /etc/pki/tls/certs/servidor-
resusta.crt

# Debian o Ubuntu
sudo apt install apache2 -y
```

```
sudo a2enmod ssl
sudo install -m 600 -o root -g root servidor/servidor.key
/etc/ssl/private/servidor-resusta.key
sudo install -m 644 servidor/servidor-san.crt /etc/ssl/certs/servidor-resusta.crt
```

Configura un sitio HTTPS con `SSLEngine on`, `SSLCertificateFile` y `SSLCertificateKeyFile` apuntando a los archivos instalados. Añade los nombres de prueba a `/etc/hosts` para resolverlos en local:

```
127.0.0.1 www.servidorresusta.test cursoresusta.test www.cursoresusta.test
```

Prueba un nombre incluido y otro que no esté en SAN. Importar `ca/ca.crt` como autoridad de confianza en el navegador es opcional y debe hacerse solo en el perfil de entorno de pruebas.

7.4. Alternativa gráfica con XCA

Como alternativa a los comandos anteriores, se puede utilizar XCA para crear una base de datos protegida, una CA raíz y una CSR de servidor. Selecciona las plantillas de CA y servidor HTTPS, utiliza SHA-256, genera una clave RSA de 4096 bits para la CA y añade los nombres DNS en SAN. Exporta el certificado público de la CA y el certificado del servidor en PEM. La clave privada del servidor solo se exportará para instalarla en el servidor de entorno de pruebas con permisos `600`.

8. Práctica 6 - Comprobación de TLS

8.1. Consultar un certificado público

1. Consulta el certificado que presenta un sitio web público. Sustituye el dominio si fuera necesario:

```
openssl s_client -connect www.example.com:443 -servername www.example.com
</dev/null 2>/dev/null \
| openssl x509 -noout -subject -issuer -dates
```

2. Identifica el sujeto, emisor y periodo de validez. Explica por qué el nombre solicitado mediante `-servername` es relevante para servidores que alojan varios sitios HTTPS.
3. Relaciona el certificado observado con la cadena de confianza, TLS y la protección de la clave privada del servidor.

8.2. Análisis pasivo con SSL Labs

Accede a [SSL Labs](#) y analiza cuatro sitios HTTPS públicos. Limita la actividad al análisis ofrecido por la web, sin realizar pruebas intrusivas. Para cada sitio, anota la calificación, versiones TLS, suites criptográficas, cadena de certificado y avisos relevantes.

Compara además el tipo de validación y la adecuación para el servicio de los sitios de BBVA, Banco Santander, El País y El Mundo. Las etiquetas DV, OV o EV no sustituyen el análisis de la configuración TLS ni garantizan la seguridad global del sitio.

9. Actividades

9.1. Conceptos y mecanismos

1. Explica la diferencia entre cifrado, hash y firma digital.
2. Compara criptografía simétrica y asimétrica: claves, rendimiento, casos de uso y principal limitación.
3. Completa la tabla:

Algoritmo o mecanismo	Tipo	Uso principal	Característica
AES			
RSA			
SHA-256			
Argon2			

10. Autoevaluación

1. ¿Qué mecanismo proporciona principalmente confidencialidad: hash, cifrado o firma digital?
2. ¿Qué clave debe mantenerse protegida en un sistema asimétrico?
3. ¿Cuál de estos es una función hash: RSA, AES, SHA-256 o ECDSA?
4. ¿Para qué se utiliza principalmente una firma digital?
5. ¿Qué elemento relaciona una identidad con una clave pública?
6. ¿Qué protocolo protege las comunicaciones HTTPS?
7. ¿Por qué no se debe utilizar ECB para datos sensibles?
8. ¿Qué aporta un *salt* al almacenamiento de contraseñas?
9. ¿Qué función cumple una CSR?
10. ¿Qué diferencia existe entre una CA raíz y una CA intermedia?
11. ¿Qué mecanismos permiten comprobar la revocación de certificados?
12. ¿Por qué un certificado autofirmado no es adecuado para un servicio público?
13. ¿Qué riesgo supone reutilizar un *nonce* con el mismo algoritmo y clave cuando el modo lo prohíbe?
14. ¿Qué amenaza representa el algoritmo de Shor para RSA y ECC?

11. Tarea evaluable única - Implementación de una PKI

Entrega un informe en PDF o Markdown con capturas propias y explicaciones redactadas con tus palabras. No incluyas claves privadas reales, contraseñas ni el contenido de archivos `.key`.

11.1. Situación

Una empresa dispone de servicios internos y necesita proteger sus comunicaciones. Como administrador de sistemas, debes diseñar e implementar una PKI mínima de entorno de pruebas.

11.2. Contenido obligatorio

1. Diagrama de la CA, el servidor, la clave privada, la CSR, el certificado firmado y el cliente.
2. Evidencias del cálculo y comprobación de un hash antes y después de modificar un fichero.
3. Creación y protección de la clave del servidor, sin mostrar su material privado.

4. Explicación de la auditoría realizada solo sobre cuentas de prueba y de las medidas que dificultan ataques contra contraseñas.
5. Creación de la CA, CSR y certificado de servidor firmado por la CA.
6. Verificación de la cadena mediante `openssl verify` e identificación de sujeto, emisor, validez y nombres SAN.
7. Evidencias de cifrado simétrico y de un mensaje cifrado y firmado con GnuPG, sin claves privadas.

12. Recursos

- [Manual de OpenSSL](#)
- [Documentación de GnuPG](#)
- [NIST: estándares de criptografía post-cuántica](#)
- [Let's Encrypt: documentación sobre certificados](#)
- [John the Ripper](#)
- [XCA](#)
- [SSL Labs](#)